CodeBuddy Agent SDK: Python to PDF Reports

Windows setup and complete example using synthetic physiotherapy data (JSON or TXT). This guide invokes the CodeBuddy-compatible agent runtime, not an existing WorkBuddy desktop conversation.

1. Prerequisites

Install Python 3.10+ from https://www.python.org/downloads/ . In PowerShell, open your project directory and run:

```
mkdir my-ai-project
cd my-ai-project
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install codebuddy-agent-sdk reportlab
```

The SDK may use a bundled CodeBuddy CLI runtime. If it cannot locate a compatible CLI, additional runtime installation/configuration may be required. The exact behavior depends on SDK version.

2. Authentication

Get a compatible API key (if your account is eligible) at https://www.codebuddy.ai/profile/keys . Set it for your current PowerShell session:

```
$env:CODEBUDDY_API_KEY = "YOUR_API_KEY"
```

Never commit credentials to source control. A desktop WorkBuddy login does not necessarily grant CodeBuddy API access. Check official SDK documentation: https://www.workbuddy.ai/docs/cli/sdk-python .

3. Project structure

```
my-ai-project/
  main.py
  data.json              # or data.txt
  reports/               # created by the script
```

4. Example physiotherapy data.json

These are invented demonstration values, not clinical measurements or prescribed targets.

```
{
  "patient_id": "PT001",
  "session_date": "2026-10-10",
  "exercise": "Standing Knee Flexion",
  "affected_leg": "right",
  "total_repetitions": 10,
  "joint_measurements": {
    "knee_flexion_degrees": [72,75,78,80,82,79,77,74,71,68],
    "hip_flexion_average_degrees": 12.5,
    "femur_internal_rotation_average_degrees": 6.2,
    "femur_external_rotation_average_degrees": 4.8
  },
  "performance": {
    "completed_repetitions": 10,
    "compensatory_movements": ["Hip hiking", "External rotation"]
  },
  "pain_score_before": 2,
  "pain_score_after": 4,
  "therapist_note": "Reduced flexion in later repetitions."
}
```

5. Complete main.py

The agent analyzes the text and writes report content. ReportLab produces two PDF files, so PDF creation does not depend on agent file-writing tools.

```
import asyncio
import json
import sys
```

```
from pathlib import Path
from xml.sax.saxutils import escape
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import getSampleStyleSheet
from codebuddy_agent_sdk import (
    query, CodeBuddyAgentOptions, AssistantMessage, TextBlock
)


def read_input(filename):
    path = Path(filename)
    if not path.is_file():
        raise FileNotFoundError(path)
    if path.suffix.lower() == ".json":
        with path.open(encoding="utf-8") as f:
            obj = json.load(f)
        return json.dumps(obj, indent=2, ensure_ascii=False)
    if path.suffix.lower() == ".txt":
        return path.read_text(encoding="utf-8")
    raise ValueError("Only .json and .txt are supported")


async def ask_agent(prompt):
    parts = []
    options = CodeBuddyAgentOptions(
        permission_mode="plan", max_turns=10
    )
    async for msg in query(prompt=prompt, options=options):
        if isinstance(msg, AssistantMessage):
            for block in msg.content:
                if isinstance(block, TextBlock):
                    parts.append(block.text)
    answer = "\n".join(parts).strip()
    if not answer:
        raise RuntimeError("Agent returned no text")
    return answer


def make_pdf(text, output_path, title):
    styles = getSampleStyleSheet()
    doc = SimpleDocTemplate(str(output_path))
    content = [Paragraph(escape(title), styles["Title"]),
               Spacer(1, 16)]
    for line in text.splitlines():
        if line.strip():
            content.append(Paragraph(
                escape(line), styles["BodyText"]
            ))
            content.append(Spacer(1, 8))
    doc.build(content)


async def generate_reports(filename):
    data = read_input(filename)
    folder = Path("reports")
    folder.mkdir(exist_ok=True)

    analysis_prompt = f"""You are a physiotherapy data analyst.
Analyze this synthetic session dataset. Explain joint angles,
repetition patterns, movement consistency, compensations,
and pain scores. Calculate useful descriptive statistics.
Separate observations from hypotheses. Do not diagnose,
prescribe treatment, or invent data. Output plain text with
clear section headings. DATA:\n{data}"""

    summary_prompt = f"""Summarize the following synthetic
physiotherapy session in plain text. Include exercise,
repetitions, key joint measurements, pain scores, and main
observations. Be concise and do not infer a diagnosis.
DATA:\n{data}"""

    analysis = await ask_agent(analysis_prompt)
    summary = await ask_agent(summary_prompt)
    analysis_path = folder / "analysis_report.pdf"
    summary_path = folder / "summary_report.pdf"
    make_pdf(analysis, analysis_path, "Analysis Report")
    make_pdf(summary, summary_path, "Summary Report")
```

```
    return analysis_path, summary_path


if __name__ == "__main__":
    if len(sys.argv) != 2:
        sys.exit("Usage: python main.py data.json")
    a, s = asyncio.run(generate_reports(sys.argv[1]))
    print("Generated:", a.resolve(), s.resolve())
```

6. Run and reuse

```
python main.py data.json
# or
python main.py data.txt
```

To call it from another async module: from main import generate_reports; analysis, summary = await generate_reports("data.json"). In a synchronous script, use asyncio.run(generate_reports("data.json")).

7. Expected output

```
reports/
  analysis_report.pdf
  summary_report.pdf
```

8. Limitations and checks

This is starter code, not a validated clinical system. It sends the entire input to the agent; large files may exceed context limits. Agent calculations and interpretations must be independently checked. ReportLab here renders basic paragraphs, not charts or Markdown. Default PDF fonts may not support all Unicode characters. Verify PDF readability and content before sharing. Protect patient information and do not send identifiable clinical data to a service without appropriate authorization and privacy safeguards.

SDK API availability and authentication were not live-tested in this environment. Verify imports and options against your installed SDK version before production use.